

ReproBot: A Multi-Agent System for Automated Scientific Paper Replication

Third Progress Report

Aleyna Kütük
aleynakutuk10@gmail.com

Engincan Varan
engincanvaran@gmail.com

Gül Korkut
serifegulkorkut@gmail.com

Göktuğ Mert Özdoğan
goekmeroz@gmail.com

inzva AI Projects #10
13.09.2026

Abstract

ReproBot is a multi-agent system that reads a machine learning paper, extracts its claims and experimental setup, generates an implementation, runs it in a sandbox, and repairs execution failures through a bounded retry loop. This report presents its first full-fidelity replications. We generalized code generation beyond image classification — the Coder now writes a plain PyTorch training loop for neural models and a scikit-learn estimator for classical ones — and evaluated on five papers whose original experiments train on a CPU, spanning five model families: a network with a margin-based loss, a tabular regression network, a kernel support vector machine, a random forest and a soft decision tree. For each we report the learning curve of the generated implementation and a comparison with the published result. Three reproductions agree closely with their papers: 0.82% against 0.87% MNIST test error, 96.63% against 96.9% SVM accuracy — with the paper’s own settings reproducing its numbers exactly — and 87.73% against 87.3% random-forest accuracy; the soft decision tree was still training at the time of writing. A housing-price network reaches a test RMSE of 4.48 against 3.02, a gap we trace to the paper’s unspecified data split. The problems met along the way, from a misprinted loss to a dead dataset link, were each resolved, three of them automatically by the retry loop. We also present the team’s first front end, a dashboard over the pipeline’s outputs, and set out the next phase: live checks on running training jobs, an automated Critic, and tighter control of the retry loop.

1 Introduction

Reproducing a machine learning paper’s reported results is mechanical in principle and laborious in practice, and most published work ships no usable implementation. ReproBot automates the process as a pipeline of specialized stages. A vision-language model converts the paper’s PDF into Markdown, reading figures as well as text. A Reader extracts five structured fields — a method summary, architecture notes, the claimed results, the hyperparameters and the data pipeline — and cross-checks them with a validation loop. A Coder turns that extraction into a self-contained training script. A Runner executes the script in a Docker container through four escalating stages: **probe**, a few optimizer steps; **smoke**, one epoch on a small slice; **capped**, five epochs on a small slice; and **full**, the paper’s own setup. When a stage fails, a small model classifies the failure, and an Orchestrator feeds that diagnosis back to the Coder and retries, within a fixed budget.

Before this phase the pipeline ran end to end but had only ever executed short checks: its CIFAR-10 target papers need weeks of CPU time per full training run. This phase produced the first full-fidelity results. Its contributions are:

Table 1: The five replication targets of this phase.

Paper	Model family	Data	Targeted claim
Tang (2013) [1]	MLP with an L2-SVM output layer	MNIST	0.87% test error
Wijaya (2023) [2]	Multi-branch dense network (Keras)	Boston Housing [3], tabular	Test RMSE 3.02
Hsu, Chang and Lin (2003) [4]	RBF support vector machine (LIBSVM [5])	svmguidel, tabular	96.9% test accuracy
Xiao, Rasul and Vollgraf (2017) [6]	Random forest (scikit-learn [7])	Fashion-MNIST	0.873 test accuracy, mean of 5 runs
Frosst and Hinton (2017) [8]	Soft decision tree	MNIST	94.45% test accuracy

- a Coder that handles any supervised task and model family, writing PyTorch for neural networks and scikit-learn for classical estimators (Section 3);
- five complete replications across five model families, each with the implementation’s learning curve and a comparison with the paper (Section 4);
- a dashboard front end over the pipeline’s outputs (Section 5);
- a concrete plan for the next phase, grounded in what the runs showed (Section 8).

2 Replication Targets

Full training of the CIFAR-10 benchmark papers is out of reach on the project’s hardware: one *Wide Residual Networks* run at the paper’s setup measured at about 22 days on the development CPU. Testing the pipeline’s actual claim — that a paper can be read, implemented, executed and checked automatically — needs only papers whose experiments are small. We chose five that train in one minute to one hour, deliberately covering different model families and data types (Table 1), and kept them apart from the CIFAR-10 set.

Two of the targets are classification networks, one a regression network, and two classical methods. Tang’s contribution is a loss function; Wijaya’s model was written in Keras; the SVM guide’s result comes from a full model-selection procedure; the Fashion-MNIST figure is an average over five runs; and the soft-tree paper states its tree depth but none of its training settings. Each property exercises a different part of the pipeline.

3 Pipeline Updates

3.1 One training script for any model family

Neural models: a plain PyTorch loop. The Coder previously emitted scripts built on HuggingFace’s **Trainer**, whose image-model interface suits neither a custom loss nor a regression head. It now writes an explicit training loop in which the optimizer, schedule and loss are all visible. When a paper was written in another framework, the Coder reproduces that framework’s defaults where the paper is silent — Keras’s Adam $\epsilon = 10^{-7}$, for example — and discloses each difference.

Classical models: scikit-learn. A support vector machine re-implemented as a PyTorch model trained by gradient descent would use a different optimizer and reach a different solution, so the Coder chooses its library by model family: SVMs, decision trees, forests, k -nearest neighbours and linear models are scikit-learn estimators with every hyperparameter passed by

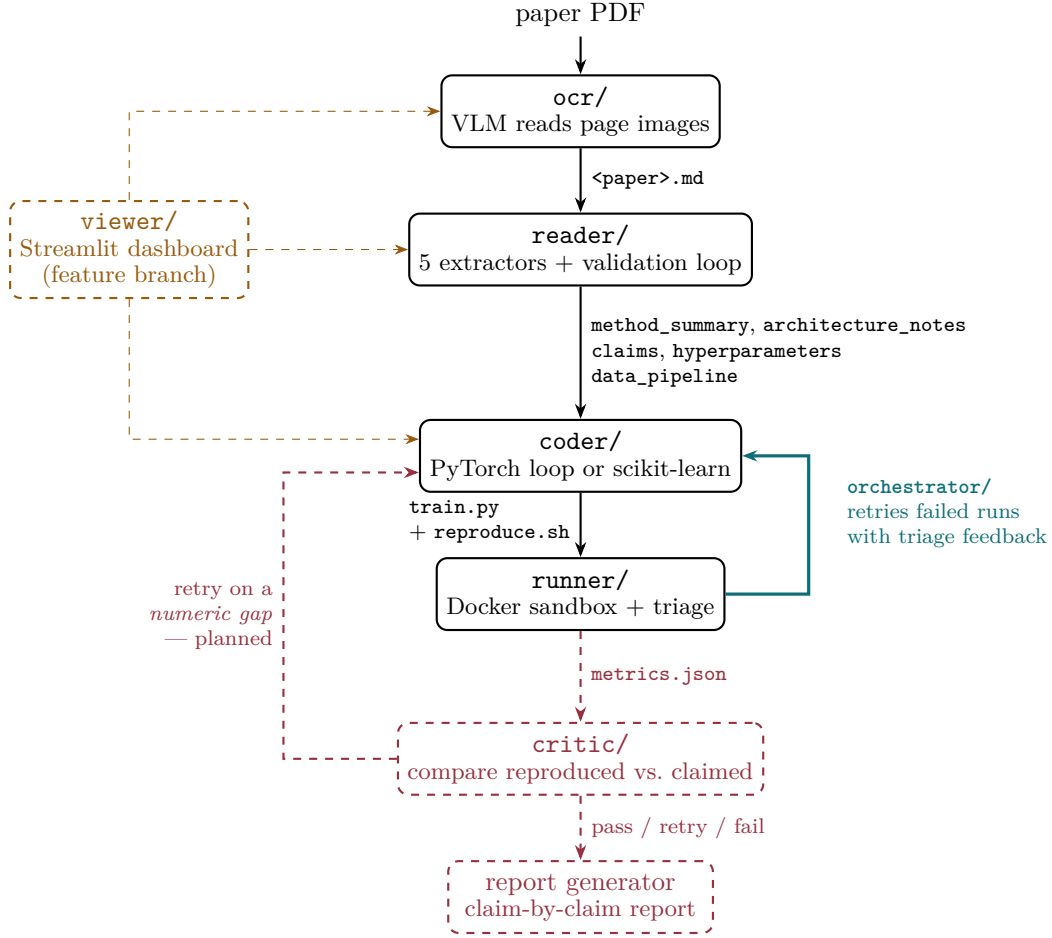


Figure 1: ReproBot on 13.09.2026. Solid components are built and verified by execution; dashed red components are planned for the next phase; the dashed amber component is on a feature branch. The Coder writes a PyTorch training loop or a scikit-learn estimator depending on the paper’s model family, and the Orchestrator retries failed runs by feeding the Runner’s diagnosis back to the Coder.

name, while networks — including trees trained by gradient descent — stay in PyTorch. Because a paper’s “default parameters” are those of the library version it used, the Coder sets paper-era defaults explicitly; scikit-learn changed several in version 0.22.

A common contract. Every script accepts the same command-line flags, so the Runner needs no per-paper logic, and writes the same metrics file: the claim’s own metric on the training and test splits, the metric’s direction, the task type, and, when the claim is an average over runs or folds, every run’s value. Data comes from stable sources — `torchvision`, OpenML or the paper’s own dataset site — and each script checks the dataset’s name and shape before training on it.

3.2 Problems encountered and how we solved them

Table 2 lists the problems the replications surfaced and the fix applied to each. Three were runtime defects in generated code that the retry loop repaired without human involvement; the rest led to permanent changes in the prompts or the pipeline, or to a documented correction.

Table 2: Problems encountered during the replications, and how each was solved.

Problem	Solution
HuggingFace Trainer silently applied gradient clipping that no paper states	Explicit PyTorch training loop; every setting written down
Re-implementing classical models in PyTorch would change their solver	Library chosen by model family; scikit-learn with paper-era defaults
A remembered dataset URL had gone dead (HTTP 403), and triage filed it as an environment fault	OpenML first; triage treats one refusing source as a fixable script error
Dataset identifiers recalled from memory can name the wrong dataset	No identifiers from memory; scripts assert the dataset’s name and shape
Tang: two unstated defaults (momentum 0.9, $C = 1.0$) together destabilized training at full scale	A six-configuration ablation isolated the pair; $C = 0.1$ trains stably
Soft tree: the paper prints its loss as the logarithm of a non-positive quantity	Implemented the evident intent, the expected cross-entropy
Runtime defects: a wrong API call (Wijaya), a label-sorted data file (SVM), an in-place autograd operation (soft tree)	Repaired automatically by the retry loop, one retry each
A dataset paper has no “own method”, so the first claims extraction was empty (Fashion-MNIST)	The validation loop flagged it; the retry extracted 26 claims
The full stage wrote its metrics under a different filename than the Runner reads	Script template corrected

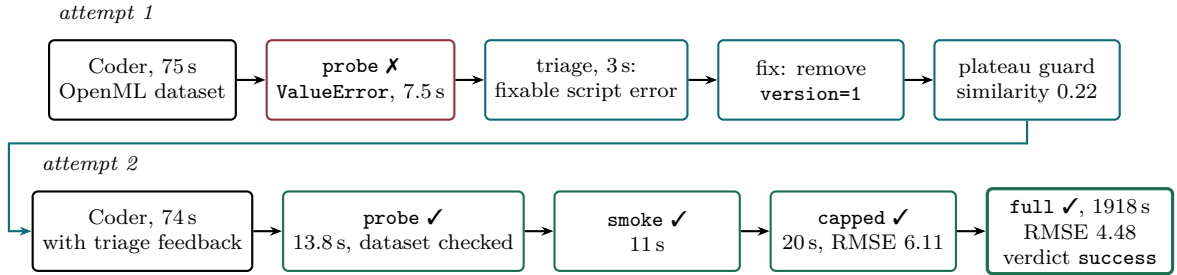


Figure 2: An automatic repair by the retry loop, on the housing-price paper. The first script failed its first check; triage diagnosed a fixable script error, the Coder regenerated the script with that feedback, and the second attempt passed every stage through the full run.

3.3 The retry loop in action

Figure 2 shows a typical automatic repair. The first generated script for the housing-price paper passed two conflicting arguments to scikit-learn’s dataset loader and stopped within seconds. Triage classified the failure as a fixable script error and proposed removing one argument; the Coder regenerated the script with that feedback, and the second attempt passed every stage through the full run. The SVM and soft-tree scripts were repaired the same way. A plateau guard, which compares each regenerated script with its predecessor to stop a loop that is not changing anything, measured similarities of 0.22 to 0.27, far from its 0.98 stopping threshold.

Table 3: Reproduced results against the published claims. Runtimes are the full stage’s wall clock on the development CPU.

Paper	Claim	Paper	ReproBot	Runtime	Agreement
Tang (2013)	MNIST test error	0.87%	0.82%	54 min	Within 0.05 points
Wijaya (2023)	Boston Housing test RMSE	3.02	4.48	32 min	Not matched; train RMSE 2.94 vs 2.69
Hsu et al. (2003)	svmguide1 test accuracy	96.9%	96.63%	73 s	Within 0.3 points; exact at the paper’s settings
Xiao et al. (2017)	Fashion-MNIST accuracy, mean of 5 runs	0.873	0.8773	3.3 min	Within 0.5 points
Frosst and Hinton (2017)	MNIST test accuracy	94.45%	(running)	—	Pending

4 Results

4.1 Overview

Table 3 compares each reproduction with its paper, and Figure 3 shows the learning curve of each generated implementation. All five scripts were generated by the Coder and executed by the Runner at the papers’ full settings. Two needed a manual correction before their final run, listed in Table 2: one unstated hyperparameter for Tang, and the misprinted loss for the soft tree.

4.2 Tang: MLP with an L2-SVM output layer

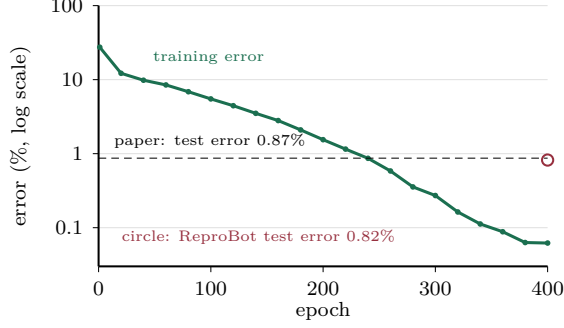
The network — PCA to 70 dimensions, two hidden layers of 512 units, and a linear SVM trained with a squared hinge loss in place of softmax — trained for the paper’s 400 epochs. Training error fell steadily to 0.06% (Figure 3a), and the test set, evaluated once at the end, gave 0.82% against the paper’s 0.87%: five test images out of 10,000. With one seed we cannot measure run-to-run variance, but for scale, the binomial standard error of an error rate this size over 10,000 images is about 0.09 points, so the result is consistent with the claim. The paper does not state its momentum or its SVM penalty C ; the first full run, with common defaults for both, became unstable, and an ablation over the unstated values showed $C = 0.1$ to train cleanly.

4.3 Wijaya: tabular regression network

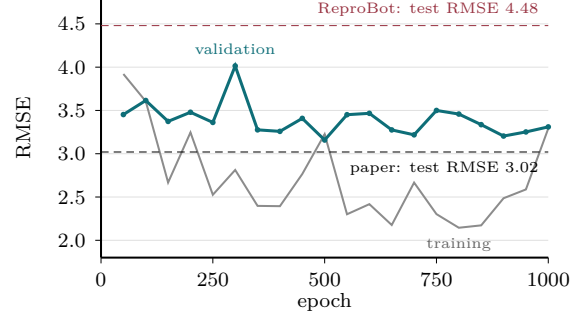
The three-level dense network, written in Keras by the paper and translated with Keras’s defaults, trained for 1000 epochs. Figure 3b shows the training and validation RMSE: the model fits within the first 150 epochs, and its validation RMSE, on 81 training rows held out as the paper describes, stays between 3.2 and 3.5 — close to the paper’s test RMSE of 3.02. The 101-row test set, however, gave 4.48, while the training RMSE of 2.94 is within 9% of the paper’s 2.69. The paper states its 405/101 split sizes but not how rows were assigned, and on a 506-row dataset two held-out subsets already disagree by more than the gap being measured; the split, not the model, is the most likely source of the difference, and several seeds would be needed to confirm it.

4.4 Hsu, Chang and Lin: RBF support vector machine

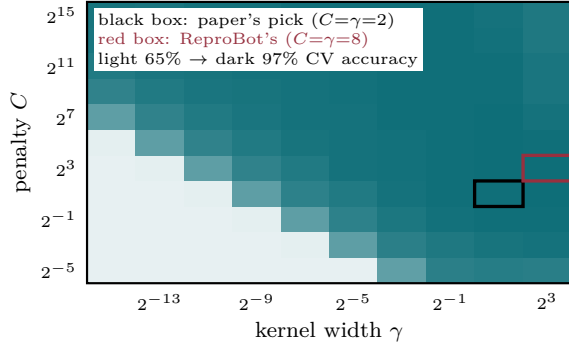
ReproBot reproduced the guide’s full procedure: scale each feature to $[-1, 1]$ with the training set’s ranges, evaluate 110 (C, γ) pairs by five-fold cross-validation, retrain with the best pair,



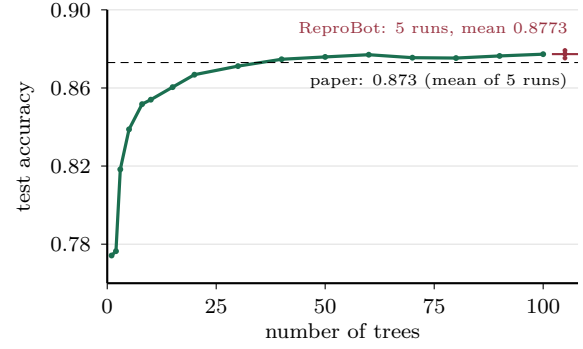
(a) MLP with L2-SVM loss, MNIST



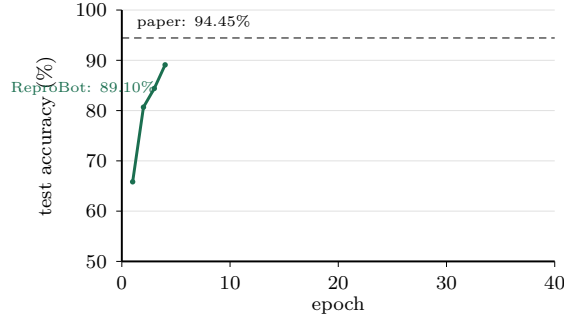
(b) Dense network, Boston Housing



(c) RBF SVM model selection, svmguide1



(d) Random forest, Fashion-MNIST



(e) Soft decision tree, MNIST

Figure 3: Learning curves of the five generated implementations at the papers' full settings, with each paper's reported result as a dashed line. (a) Training error per 20 epochs; the circle is the reproduced test error. (b) Training and validation RMSE per 50 epochs (epoch 1 is off the scale). (c) Five-fold cross-validation accuracy over the SVM's (C, γ) grid, with the paper's and ReproBot's selected settings outlined. (d) Test accuracy as trees are added, for the first of five runs, with all five runs' final accuracies at the right. (e) Test accuracy per epoch. Panels (c) and (d) were computed with the generated scripts' own configuration and code; all other points are read directly from the run logs.

Table 4: svmguide1 test accuracy: the paper’s LIBSVM results against scikit-learn’s SVC at the same settings.

Setting	Paper	scikit-learn
Unscaled features, default parameters	66.925%	66.925%
Scaled to $[-1, 1]$, default parameters	96.15%	96.150%
Scaled, $C = 2$, $\gamma = 2$	96.875%	96.875%
Scaled, grid search (ReproBot: $C = \gamma = 8$)	—	96.625%

Table 5: Reader output across nine papers; the first four are CIFAR-10 papers. *Comp.* counts architecture components; *own/eq* equations marked as the paper’s own over equations captured; *Gaps* explicitly recorded unstated details; *Flags* unresolved validation flags.

Paper	Claims	Hyp.	Data	Comp.	own/eq	Gaps	Flags
Network In Network	12	17	4	7	1/3	12	4
All Convolutional Net	17	13	4	10	2/6	11	5
Deep Residual Learning	66	28	6	10	2/3	7	8
Wide Residual Networks	61	20	4	11	1/1	11	6
Tang (2013)	15	23	3	10	4/11	12	5
Wijaya (2023)	14	10	1	8	2/2	8	5
Hsu et al. (2003)	3	19	8	2	0/6	5	0
Xiao et al. (2017)	26	14	1	0	0/0	3	0
Frosst and Hinton (2017)	8	11	3	4	5/5	10	4

and test once. Figure 3c shows the cross-validation surface. A broad ridge of settings scores above 96%; ReproBot’s search picked $C = \gamma = 8$ at 96.99%, the paper’s picked $C = \gamma = 2$ at 96.89%, and the two are within a tenth of a point. The test accuracy was 96.625% against the paper’s 96.875%. Because scikit-learn’s SVC wraps LIBSVM, we also evaluated it directly at each of the paper’s reported settings (Table 4): all three test accuracies match the paper exactly. The implementation is therefore exact, and the small remaining gap comes only from which near-equal setting the randomly folded search selects.

4.5 Xiao, Rasul and Vollgraf: random forest

The Fashion-MNIST benchmark reports the mean test accuracy of five runs with shuffled training data; ReproBot’s script performed all five with the paper’s configuration (100 trees, entropy criterion, maximum depth 100). The runs gave between 0.8753 and 0.8792, with a mean of 0.8773 against the paper’s 0.873. Figure 3d shows test accuracy as trees are added: it rises quickly to about 0.87 by 30 trees and then flattens, so the paper’s choice of 100 trees sits on the plateau. The reproduction is within half a point of the claim; the scikit-learn version the paper used is not stated and may account for the remaining difference.

4.6 Frosst and Hinton: soft decision tree

The full run was in progress at the time of writing.

4.7 Extraction across paper types

The Reader ran unchanged on all five papers. Table 5 compares them with the four CIFAR-10 papers it had processed earlier. Every targeted claim was extracted. For both classical-ML papers, validation converged with no open flags, a first for the pipeline; for the equation-heavy soft-tree paper, all five equations were correctly identified as the paper’s own.

4.8 Cost

A Coder call consumes about 27,000 input and 9,500 output tokens and takes 75s; a triage call about 2,400 input and 240 output tokens, in 3s. Compute dominates: the full runs took from 73s (SVM) to 54 minutes (Tang). Running two containers at once roughly doubled wall clock, because each claims every CPU core, so runs are best scheduled one at a time.

5 A Front End: The Pipeline Viewer

In parallel, the team built ReproBot’s first user interface: `viewer/`, a Streamlit dashboard developed on a feature branch. Its main page lists every paper with its pipeline status — whether OCR, Reader and Coder output exist, and how many validation flags remain. Selecting a paper opens tabs for its method summary, architecture notes, claims, hyperparameters, data pipeline, validation flags, raw extraction, the generated training script with its `reproduce.sh`, and the OCR Markdown, so a reviewer can compare what was extracted with what was generated without opening files. An “Import a paper” section accepts a PDF, with an optional page limit for cheap trials, and buttons run Reader extraction and code generation by calling each stage’s own entry point, so the viewer cannot drift from the command-line pipeline. The branch was verified with strict type checking and a headless run that rendered every tab. Its next steps are to display Runner and Orchestrator output — the learning curves and comparisons of this report — and to merge into the main pipeline as the basis of the project’s final demonstration.

6 Lessons

Watching a run beats waiting for it. The two training problems of this phase — Tang’s instability and the soft tree’s reversed loss — were both identified by reading the container’s log while the run was in progress, long before it would have finished. A script that exits normally has not necessarily learned; the Runner should read these logs itself.

Some claims can be matched exactly, others only within variance. The SVM guide reproduces to three decimal places at the paper’s settings, while the housing-price result depends on an unrecorded random split. A single tolerance cannot serve both, so the Critic’s must depend on how a claim was produced.

Under-specified papers are reproduced through disclosed guesses. The soft-tree script had to choose about a dozen unstated values and recorded each one; Tang’s instability came from a combination of two individually reasonable guesses. Disclosure makes such choices visible and correctable, which is what allowed both to be fixed.

Classical and neural models need different checks. For the random forest and the SVM, the `smoke` and `capped` stages gave identical numbers, since they differ only in epochs; for models without epochs, a ladder over training-set size would be more informative.

Limitations. The comparisons were made by hand, mostly from single runs; two of the five reproductions needed a manual correction; the CIFAR-10 benchmark still needs GPU compute; and Boston Housing includes a feature derived from the proportion of Black residents per town [3], which we used only because the replicated paper did.

7 Progress Against the Plan

The project was planned over four months: the Reader first; then the Coder, sandbox and a skeleton Orchestrator; then the Critic, a real retry loop and the Report Generator; and finally evaluation, ablation and a demonstration. Table 6 compares the work to date with that plan. The retry loop arrived ahead of plan; the Critic and Report Generator were deferred in favour of first obtaining the measurable results they need as input.

Table 6: Status against the project’s four-month plan. ✓ done; ∼ partial or in a different form; ✗ not started.

Phase	Planned	Status
Month 1	PDF ingestion; Reader schema; shared-memory object	✓ Complete; five fields
Month 2	Coder and Docker sandbox; Orchestrator skeleton with a single retry	✓ Exceeded: bounded retry loop with triage-driven feedback
Month 3	Critic verdict logic	✗ Next phase; inputs now exist
	Real retry loop with targeted feedback	✓ Three real defects repaired
	Report Generator	✗ Next phase
	Single-shot vs. iterative ablation	∼ Three cases fail single-shot and run after one retry
Month 4	Expansion towards 20 papers	∼ Nine papers read; five replicated at full fidelity
	Demonstration interface	∼ Streamlit viewer on a feature branch

8 Future Work

Live check-ups in the Runner. Read the container’s log during every run, parse the per-epoch metrics, and stop early with a clear verdict when training is not progressing: a loss that does not move, non-finite values, accuracy at or below chance, or a loss outside its possible range. Generated scripts will also write their learning curves to a structured file, so the checks, the Critic and the reports can use them directly.

The Critic agent. Compare each reproduced value with the claim using the metric’s direction and a tolerance matched to the claim: near-exact for deterministic procedures, a seed-variance band for stochastic training, and the reported spread when a claim is itself an average. Its verdict — pass, retry or fail, with a reason — closes the loop the project is built around.

Loop controls. Let the Orchestrator patch the faulty lines of a script instead of regenerating it, as in AutoReproduce [9]; keep unstated hyperparameters fixed across retries unless the feedback concerns them; abort a stage as soon as its live check fails; enforce per-paper time and cost budgets; and record any human correction in the run’s state.

Broader coverage. Adapt the stage ladder to models without epochs; check the domain of a paper’s own equations before implementing them; run the targets already selected — gradient boosting with XGBoost, LightGBM and CatBoost, a graph convolutional network, a text CNN and a temporal convolutional network; and secure GPU compute for the CIFAR-10 benchmark.

9 Conclusion

ReproBot now carries a paper from PDF to a running implementation in the right library, at the paper’s own scale, for neural and classical models alike. The problems met on the way were resolved, and they point clearly at the next phase: a Runner that watches training as it happens, a Critic that judges each result with the right tolerance, and a retry loop that changes only what it must.

References

- [1] Tang, Y. (2013). Deep Learning using Linear Support Vector Machines. *ICML 2013 Workshop on Challenges in Representation Learning*. arXiv:1306.0239.

- [2] Wijaya, R. (2023). Multi Level Dense Layer Neural Network Model for Housing Price Prediction. *arXiv:2310.08133*.
- [3] Harrison, D., & Rubinfeld, D. L. (1978). Hedonic housing prices and the demand for clean air. *Journal of Environmental Economics and Management*, 5(1), 81–102.
- [4] Hsu, C.-W., Chang, C.-C., & Lin, C.-J. (2003). A Practical Guide to Support Vector Classification. Technical report, Department of Computer Science, National Taiwan University (version updated 2025).
- [5] Chang, C.-C., & Lin, C.-J. (2011). LIBSVM: A library for support vector machines. *ACM Transactions on Intelligent Systems and Technology*, 2(3), 27.
- [6] Xiao, H., Rasul, K., & Vollgraf, R. (2017). Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms. *arXiv:1708.07747*.
- [7] Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- [8] Frosst, N., & Hinton, G. (2017). Distilling a Neural Network Into a Soft Decision Tree. *arXiv:1711.09784*.
- [9] Zhao, X., Sun, M., Wang, W., et al. (2025). AutoReproduce: Automatic AI Experiment Reproduction with Paper Lineage. *arXiv:2505.20662*.